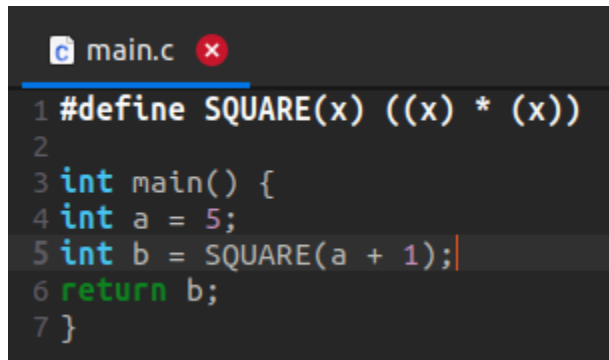
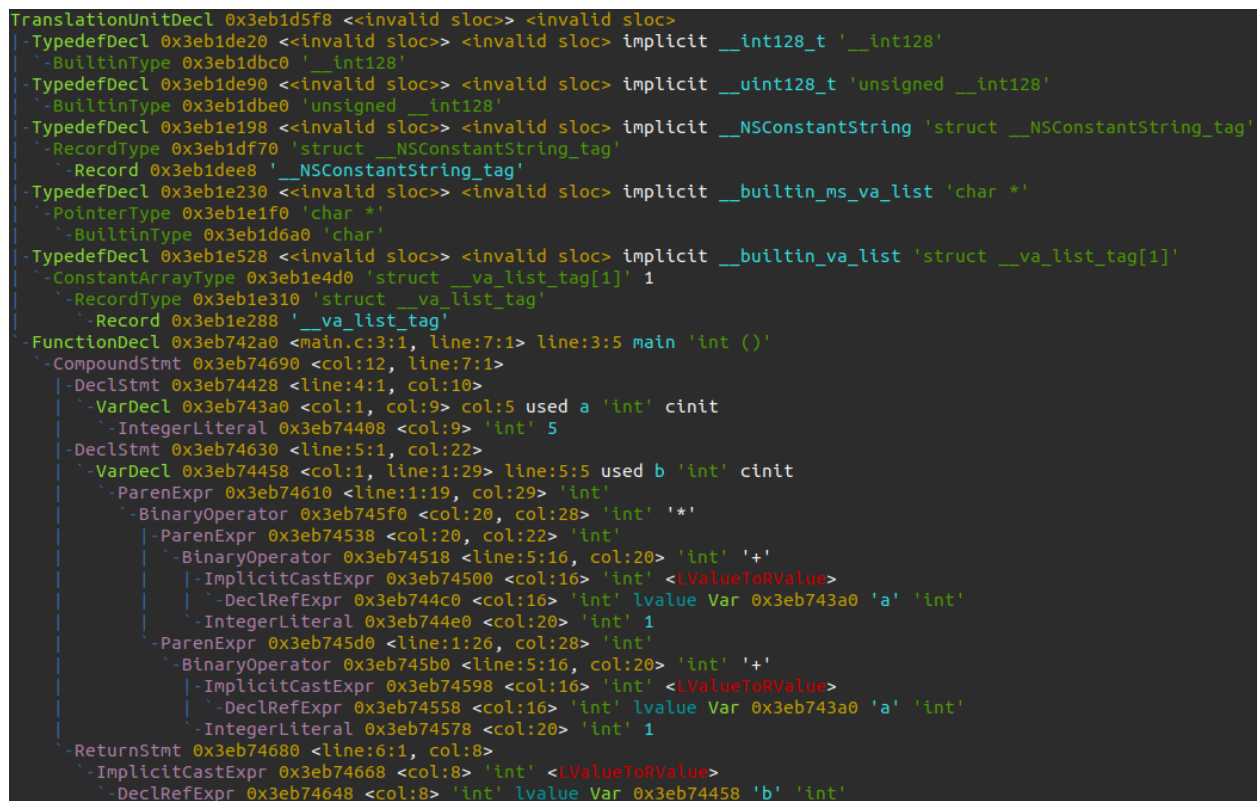


2 часть



```
main.c
1 #define SQUARE(x) ((x) * (x))
2
3 int main() {
4 int a = 5;
5 int b = SQUARE(a + 1);
6 return b;
7 }
```

Рисунок 1 – Исходный код



```
TranslationUnitDecl 0x3eb1d5f8 <<invalid sloc>> <invalid sloc>
|-TypedefDecl 0x3eb1de20 <<invalid sloc>> <invalid sloc> implicit __int128_t '__int128'
|  |-BuiltInType 0x3eb1dbc0 '__int128'
|  |-TypedefDecl 0x3eb1de90 <<invalid sloc>> <invalid sloc> implicit __uint128_t 'unsigned __int128'
|  |  |-BuiltInType 0x3eb1dbe0 'unsigned __int128'
|  |-TypedefDecl 0x3eb1e198 <<invalid sloc>> <invalid sloc> implicit __NSConstantString 'struct __NSConstantString_tag'
|  |  |-RecordType 0x3eb1df70 'struct __NSConstantString_tag'
|  |  |  |-Record 0x3eb1dee8 '__NSConstantString_tag'
|  |-TypedefDecl 0x3eb1e230 <<invalid sloc>> <invalid sloc> implicit __builtin_ms_va_list 'char *'
|  |  |-PointerType 0x3eb1e1f0 'char *'
|  |  |  |-BuiltInType 0x3eb1d6a0 'char'
|  |-TypedefDecl 0x3eb1e528 <<invalid sloc>> <invalid sloc> implicit __builtin_va_list 'struct __va_list_tag[1]'
|  |  |-ConstantArrayType 0x3eb1e4d0 'struct __va_list_tag[1]' 1
|  |  |  |-RecordType 0x3eb1e310 'struct __va_list_tag'
|  |  |  |  |-Record 0x3eb1e288 '__va_list_tag'
|-FunctionDecl 0x3eb742a0 <main.c:3:1, line:7:1> line:3:5 main 'int ()'
|  |-CompoundStmt 0x3eb74690 <col:12, line:7:1>
|  |  |-DeclStmt 0x3eb74428 <line:4:1, col:10>
|  |  |  |-VarDecl 0x3eb743a0 <col:1, col:9> col:5 used a 'int' cinit
|  |  |  |  |-IntegerLiteral 0x3eb74408 <col:9> 'int' 5
|  |  |-DeclStmt 0x3eb74630 <line:5:1, col:22>
|  |  |  |-VarDecl 0x3eb74458 <col:1, line:1:29> line:5:5 used b 'int' cinit
|  |  |  |  |-ParenExpr 0x3eb74610 <line:1:19, col:29> 'int'
|  |  |  |  |  |-BinaryOperator 0x3eb745f0 <col:20, col:28> 'int' '*'
|  |  |  |  |  |  |-ParenExpr 0x3eb74538 <col:20, col:22> 'int'
|  |  |  |  |  |  |  |-BinaryOperator 0x3eb74518 <line:5:16, col:20> 'int' '+'
|  |  |  |  |  |  |  |  |-ImplicitCastExpr 0x3eb74500 <col:16> 'int' <LValueToRValue>
|  |  |  |  |  |  |  |  |  |-DeclRefExpr 0x3eb744c0 <col:16> 'int' lvalue Var 0x3eb743a0 'a' 'int'
|  |  |  |  |  |  |  |  |  |  |-IntegerLiteral 0x3eb744e0 <col:20> 'int' 1
|  |  |  |  |  |  |  |-ParenExpr 0x3eb745d0 <line:1:26, col:28> 'int'
|  |  |  |  |  |  |  |  |-BinaryOperator 0x3eb745b0 <line:5:16, col:20> 'int' '+'
|  |  |  |  |  |  |  |  |  |-ImplicitCastExpr 0x3eb74598 <col:16> 'int' <LValueToRValue>
|  |  |  |  |  |  |  |  |  |  |-DeclRefExpr 0x3eb74558 <col:16> 'int' lvalue Var 0x3eb743a0 'a' 'int'
|  |  |  |  |  |  |  |  |  |  |  |-IntegerLiteral 0x3eb74578 <col:20> 'int' 1
|  |  |-ReturnStmt 0x3eb74680 <line:6:1, col:8>
|  |  |  |-ImplicitCastExpr 0x3eb74668 <col:8> 'int' <LValueToRValue>
|  |  |  |  |-DeclRefExpr 0x3eb74648 <col:8> 'int' lvalue Var 0x3eb74458 'b' 'int'
```

Рисунок 2 – Получение AST

Видна работа макроса. Вместо вызова в коде присутствует вычисляемое выражение: $((a+1) * (a+1))$, где оба операнда умножения представляют собой $(a+1)$.

```

1; ModuleID = 'main.c'
2 source_filename = "main.c"
3 target_datalayout = "e-m:e-p270:32:32-p271:32:32-p272:64:64-i64:64-f80:128-n8:16:32:64-S128"
4 target_triple = "x86_64-pc-linux-gnu"
5
6; Function Attrs: noline nounwind optnone uwtable
7 define dso_local i32 @main() #0 {
8   %1 = alloca i32, align 4
9   %2 = alloca i32, align 4
10  %3 = alloca i32, align 4
11  store i32 0, i32* %1, align 4
12  store i32 5, i32* %2, align 4
13  %4 = load i32, i32* %2, align 4
14  %5 = add nsw i32 %4, 1
15  %6 = load i32, i32* %2, align 4
16  %7 = add nsw i32 %6, 1
17  %8 = mul nsw i32 %5, %7
18  store i32 %8, i32* %3, align 4
19  %9 = load i32, i32* %3, align 4
20  ret i32 %9
21 }
22
23 attributes #0 = { noline nounwind optnone uwtable "frame-pointer"="all" "min-legal-vector-width"="0" "no-trapping-math"="true" "stack-protector-buffer-size"="8" "target-cpu"="x86-64" "target-features"="+cx8,+fxsr,+mmx,+sse,+sse2,+x87" "tune-cpu"="generic" }
24
25 !llvm.module.flags = !{!0, !1, !2, !3, !4}
26 !llvm.ident = !{!5}
27
28 !0 = !{i32 1, !"wchar_size", i32 4}
29 !1 = !{i32 7, !"PIC Level", i32 2}
30 !2 = !{i32 7, !"PIE Level", i32 2}
31 !3 = !{i32 7, !"uwtable", i32 1}
32 !4 = !{i32 7, !"frame-pointer", i32 2}
33 !5 = !{!"Ubuntu clang version 14.0.0-1ubuntu1.1"}

```

Рисунок 3 – Оптимизация IR (O0)

```

1; ModuleID = 'main.c'
2 source_filename = "main.c"
3 target_datalayout = "e-m:e-p270:32:32-p271:32:32-p272:64:64-i64:64-f80:128-n8:16:32:64-S128"
4 target_triple = "x86_64-pc-linux-gnu"
5
6; Function Attrs: mustprogress norecurse nosync nounwind readnone uwtable willreturn
7 define dso_local i32 @main() local_unnamed_addr #0 {
8   ret i32 36
9 }
10
11 attributes #0 = { mustprogress norecurse nosync nounwind readnone uwtable willreturn "frame-pointer"="none" "min-legal-vector-width"="0" "no-trapping-math"="true" "stack-protector-buffer-size"="8" "target-cpu"="x86-64" "target-features"="+cx8,+fxsr,+mmx,+sse,+sse2,+x87" "tune-cpu"="generic" }
12
13 !llvm.module.flags = !{!0, !1, !2, !3}
14 !llvm.ident = !{!4}
15
16 !0 = !{i32 1, !"wchar_size", i32 4}
17 !1 = !{i32 7, !"PIC Level", i32 2}
18 !2 = !{i32 7, !"PIE Level", i32 2}
19 !3 = !{i32 7, !"uwtable", i32 1}
20 !4 = !{!"Ubuntu clang version 14.0.0-1ubuntu1.1"}

```

Рисунок 4 – Оптимизация IR (O2)

Выражение вычислено на этапе компиляции. Видно, что вместо вычислений, возвращается константа: ret i32 36.

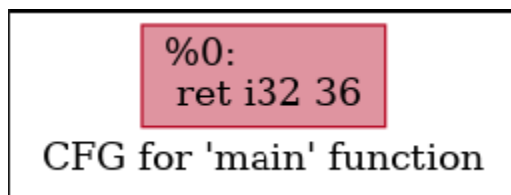


Рисунок 5 – CFG main

```

1; ModuleID = 'main.c'
2source_filename = "main.c"
3target datalayout = "e-m:e-p270:32:32-p271:32:32-p272:64:64-i64:64-f80:128-n8:16:32:64-S128"
4target triple = "x86_64-pc-linux-gnu"
5
6; Function Attrs: noline nounwind optnone uwtable
7define dso_local i32 @main() #0 {
8    %1 = alloca i32, align 4
9    %2 = alloca i32, align 4
10   %3 = alloca i32, align 4
11   store i32 0, i32* %1, align 4
12   store i32 5, i32* %2, align 4
13   %4 = load i32, i32* %2, align 4
14   %5 = call i32 @square(i32 noundef %4)
15   store i32 %5, i32* %3, align 4
16   %6 = load i32, i32* %3, align 4
17   ret i32 %6
18 }
19
20declare i32 @square(i32 noundef) #1
21
22attributes #0 = { noline nounwind optnone uwtable "frame-pointer"="all" "min-legal-vector-width"="0" "no-trapping-math"="true" "stack-protector-buffer-size"="8" "target-cpu"="x86-64" "target-features"="+cx8,+fxsr,+mmx,+sse,+sse2,+x87" "tune-cpu"="generic" }
23attributes #1 = { "frame-pointer"="all" "no-trapping-math"="true" "stack-protector-buffer-size"="8" "target-cpu"="x86-64" "target-features"="+cx8,+fxsr,+mmx,+sse,+sse2,+x87" "tune-cpu"="generic" }
24
25!llvm.module.flags = !{!0, !1, !2, !3, !4}
26!llvm.ident = !{!5}
27
28!0 = !{i32 1, !"wchar_size", i32 4}
29!1 = !{i32 7, !"PIC Level", i32 2}
30!2 = !{i32 7, !"PIE Level", i32 2}
31!3 = !{i32 7, !"uwtable", i32 1}
32!4 = !{i32 7, !"frame-pointer", i32 2}
33!5 = !{!"Ubuntu clang version 14.0.0-1ubuntu1.1"}

```

Рисунок 6 – Оптимизация IR (O0) для inline-функции

Вывод: отличие макроса от inline-функции:

- Нет отдельного объявления функции;
- Нет вызова функции;
- Умножение (mul) в макросе находится в main.c, у функции нет;
- У макроса больше инструкции, нежели у функции;